LangIndex

A Deterministic Software Adoption Index and Oracle Protocol

Nael Vask | November 2026

Genesis Hash: 87be4735c1f0c9f85c5e2822b893c097dfa9690ccd4bfa95374036b537090def

ABSTRACT

Software adoption metrics are currently fragmented across private analytics providers and ad-hoc surveys that lack verifiable data integrity, formal settlement guarantees, and open economic mechanisms. We present LangIndex, an open protocol that deterministically converts code commit frequency and package registry download telemetry into continuous price indices across major programming languages. Price formation is decoupled from secondary market speculation: positions track an exogenous adoption curve governed by mathematical formulas rather than internal order book pressure. We define a multi-stage noise suppression pipeline to eliminate burst manipulation, formalize a verifiable genesis anchor, introduce an incentive token (DEK) for decentralized oracle verification, and specify liquidity dynamics for deterministic index exposure.

1. Introduction

Traditional financial contracts reference assets whose valuations are established through continuous double auctions. In contrast, economic exposure to software ecosystems has historically been indirect, typically approximated via equity in platform monopolies or cloud providers.

Existing indices such as TIOBE, PYPL, and the GitHub Octoverse report adoption estimates via proprietary weighting schemes. These implementations exhibit three structural limitations:

- **Opaque methodology:** Survey weights and search engine filters are adjusted unilaterally without cryptographic verification.
- **Manipulation vulnerability:** Raw search volume queries and unweighted repository counts are susceptible to cheap sybil generation.
- **Lack of financial utility:** Published scores are informational tables with no settlement architecture or hedge mechanisms for developer tooling teams.

LangIndex resolves these constraints by implementing a transparent, deterministic aggregation pipeline where verified source commits, package pull telemetry, and moving-average filters produce continuous, tamper-evident indices.

2. Telemetry Ingestion and Decentralized Source Vectors

The protocol samples software activity daily across three orthogonal vector classes:

- **Active Code Production** (Decentralized and Public Repositories): Measured through version control state across GitHub, Codeberg, and decentralized peer-to-peer git protocols (Radicle). Inquiries enforce non-fork parameters (fork:false and tree-hash uniqueness) to eliminate mirrored or mechanically cloned source trees. Historical permanence is audited against Software Heritage persistent identifiers (SWHIDs).

- **Downstream Consumption:** Sampled through daily package registry pull events across PyPI (Python), npm (JavaScript and TypeScript), and crates.io (Rust). Downstream consumption verifies that pushed source code corresponds to functional library usage rather than hollow commit generation.
- **Independent Mirror Verification:** Telemetry nodes ingest direct git bundle diffs from autonomous mirrors, preventing reliance on any single commercial API gateway.

Let R(i,t) denote verified repository push counts for language i on day t. Let D(i,t) denote registry download volume for package sets associated with language i.

For languages with active registry telemetry, composite observation C(i,t) is evaluated as:

```
C(i,t) = w_repo * R(i,t) + w_pkg * (gamma * D(i,t))
```

where w_repo = 0.65, w_pkg = 0.35, and gamma = 10^-4 represents a normalization scalar aligning download counts with repository magnitudes. For languages without programmatic registry endpoints, w_repo = 1.0.

3. Mathematical Formulation and Index Mechanics

3.1 Anti-Spam Median Truncation

To suppress synthetic spam floods (e.g., scripted repository generation by bots), raw values are clipped against their trailing fourteen-day history:

```
C'(i,t) = min( C(i,t), kappa * median({ C(i, t-k) : k=1..14 }) )
```

where kappa = 2.0. An attacker generating a 10x surge within a single 24-hour cycle has all activity beyond 2.0x discarded automatically.

3.2 Sybil Haircut

Before network decentralization reaches a threshold of independent multi-party oracles, an uncertainty haircut factor alpha = 0.85 scales all inputs:

```
C~(i,t) = alpha * C'(i,t)
```

3.3 Moving Window Filtering

To remove cyclical weekend variance (developer output drops 40% to 60% on Saturdays and Sundays), values pass through a 7-day rolling window:

```
A(i,t) = (1/7) * sum( C~(i, t-k), k=0..6 )
```

3.4 Relative Market Share and Base Normalization

The ecosystem proportion S(i,t) for language i on day t is:

```
S(i,t) = A(i,t) / sum( A(j,t), j=1..M )
```

where M is the cardinality of tracked assets. The index price P(i,t) is established relative to initial base day t=0:

```
P(i,t) = 100.0 * S(i,t) / S(i,0)
```

By construction, P(i,0) = 100.00 for all assets. Price changes reflect relative growth or decline against the aggregate ecosystem baseline. Global shifts in total developer population cancel out in the denominator of S(i,t).

4. Market Microstructure and Liquidity Control

A fundamental distinction exists between an adoption index and a floating supply token. In standard asset markets, purchasing an asset directly exerts upward pressure on its price. In LangIndex, trade volume never directly modifies P(i,t):

```
P(i,t) = g( Code Commits, Package Downloads )
```

This model prevents market cornering and pump-and-dump manipulation. A trader executing a million-DEK order cannot move the adoption price by a single pip.

4.1 Native DEK-Denominated Settlement Architecture

Positions represent synthetic contracts for difference (CFD) denominated and settled natively in DEK utility tokens. The protocol enforces native closed-loop accounting:

- **100% Native Solvency Guarantee:** All collateral deposits, position values, unrealized gains, and margin settlements occur strictly in DEK units. The protocol carries zero foreign currency liabilities.
- **Aggregate Open Interest Caps:** Exposure on any single language index is bounded at OI_max = 250,000 DEK. If this ceiling is reached, additional long positions are blocked until reserves expand.
- **Anti-Whale Distribution Limits:** No single identity address may command more than theta = 15% of the asset exposure ceiling (37,500 DEK), preventing single-actor monopolization.
- **Dynamic Funding Rate Balancing:** F(i,t) = rho * (OI_long - OI_short) / OI_total. When long demand skews the book, longs pay the delta directly to shorts, protecting the treasury against systemic payout drain.
- **Virtual AMM Spread:** Spread(i,t) = base_spread + lambda * OrderSize / PoolDepth. This spread accrues to the protocol reserve, neutralizing toxic order flow.

5. The DEK Protocol Incentive Token

While index assets (PY, RS, TS) measure external software engineering activity, network coordination requires an internal utility and governance unit. We define the DEK token.

5.1 Token Utility and Distribution

DEK is emitted purely via functional participation:

- **Oracle Validation Rewards:** Nodes submitting valid telemetry matching the consensus median receive a fixed emission of 50 DEK per daily block.
- **Liquidity and Trade Volume Rebates:** Traders receive 10 basis points (0.10%) of executed volume in newly minted DEK tokens.
- **Total Supply Hard Cap:** Total DEK issuance is capped at 20,502,006 units. Emission halving occurs every 730 blocks (approximately two calendar years).

5.2 Deflationary Sinks

A minimum of 50% of all vAMM spread fees collected by the exchange core are routed to a verifiable burn address (0x000000000000000000000000000000000000dead), creating continuous deflationary pressure as protocol volume scales.

6. Decentralized Oracle Verification and Proof of Verification Work

6.1 Phased Architecture and Current Implementation Status

LangIndex follows a staged deployment model. Transparent disclosure of implementation state is a core architectural principle:

- **Phase 1 (Current):** Reference node implementation with a local ACID SQLite ledger. DEK accounting is maintained in the node database. There is no external smart contract or on-chain settlement. User balances represent paper-trading sandbox positions for indexing demonstration. This is functionally equivalent to how centralized exchanges maintain user accounts off-chain prior to withdrawal settlement.
- **Phase 2 (Target):** Multi-node consensus with on-chain settlement (e.g. Bitcoin sidechain or EVM L2). Validator block rewards and trade settlements will be finalized to smart contract state. All Phase 1 cryptographic primitives (BIP-39 derivation, Ed25519 signing, Merkle trees, PoW puzzles) are designed to map directly into Phase 2 on-chain verification.

6.2 Telemetry Verification Mechanics

Nodes do not merely query public search counters. Telemetry verification requires executing cryptographic integrity checks over active repositories:

- **Commit Tree Traversal:** A node samples active repositories, fetches the latest commit object via the GitHub API, and extracts the commit SHA and tree SHA directly.
- **Non-Fork Validation:** The node verifies through git parent graphs that pushed trees are not mechanical mirrors or automated script duplicates.
- **Package Hash Matching:** For registry metrics, nodes verify package archive SHA-256 digests against published registry index manifests.

6.3 Deterministic Sharded Sampling

Global developer activity generates approximately 300,000 repository updates daily. Requiring an ordinary participant to pull git trees for all repositories would demand 20-50 GB of bandwidth daily. LangIndex resolves this via Deterministic Sharded Sampling:

- **Seed Derivation:** Seed(u) = SHA-256(Address(u) || BlockHash(t-1))
- **Shard Allocation:** Using Seed(u) as a cryptographically secure PRNG, the network assigns node u a deterministic shard from the candidate pool. In Phase 1, the shard size is K = 10 representative repositories across tracked languages, scaling toward K = 200 as the validator pool expands.
- **Bandwidth Neutrality:** Verifying the shard requires negligible data transfer per 24-hour epoch, completing in seconds on an ordinary residential connection.

6.4 Proof of Tree Object Possession (PoTOP)

Lazy nodes attempt to claim rewards without performing work by copying valid count vectors from honest nodes and re-signing the stolen payload. LangIndex eliminates this via PoTOP:

- **Cryptographic Tree Challenge:** For every repository r in the node shard, the verifier extracts the raw git tree hash H_tree(r) and parent commit hash H_parent(r) directly from the commit object.
- **Identity-Bound HMAC Leaf Construction:** M(u,r) = HMAC-SHA256(K_node(u), RepoID(r) || H_tree(r) || H_parent(r) || EpochDay(t)) where K_node(u) = SHA-256(PrivateKey(u)).
- **Merkle Proof Root:** The node constructs a balanced binary Merkle tree across all leaf HMACs and submits the root alongside its signed telemetry vector.

A lazy node cannot copy another node's Merkle root or leaves because M(u,r) incorporates the submitter's private key over raw git commit tree blobs. Submitting an invalid or synthesized root results in an immediate

100 DEK stake slash.

6.5 Consensus Verification Gate

The reward minting engine enforces a mandatory three-tier consensus verification gate before any DEK is credited:

- **1. Proof-of-Telemetry Existence:** A valid verified_proofs record must exist in the attestation ledger for the claiming node and epoch day.
- **2. Ed25519 Signature Verification:** The stored cryptographic signature on the proof payload is mathematically validated against the node public key using crypto_engine.verify_signature().
- **3. Proof-of-Work Verification:** The submitted PoW nonce must satisfy the target difficulty (16 leading zero bits) evaluated by pow_engine.verify_pow_proof().

If any of these three conditions is not satisfied, the reward transaction is rejected and zero tokens are minted.

6.6 Mining Difficulty and Dynamic Retargeting

Each adoption block corresponds to exactly one 24-hour UTC window (86,400 seconds). Every 14 blocks, the protocol retargets proof difficulty based on active participating nodes.

6.7 ASIC Resistance

LangIndex PoVW is intrinsically ASIC-resistant due to three structural bottlenecks: (1) live network socket handshakes across distributed mirrors cannot be bypassed by pure compute hardware; (2) constructing commit Merkle graphs requires traversing git object storage in RAM; (3) nodes must manage authentic network credentials and rotational peers.

6.8 Post-Quantum Cryptographic Resilience

Symmetric algorithms and hash functions like SHA-256 retain 128 bits of security against Grover's quantum search algorithm. Node identity signatures are architected to transition from Ed25519 to lattice-based signatures (NIST FIPS 204 / ML-DSA, formerly CRYSTALS-Dilithium) or stateful hash-based signatures (LMS/XMSS) prior to quantum hardware maturity.

7. Wallet Security and Key Management

7.1 BIP-39 Key Derivation

Node keys and identities are derived from standard 12-word BIP-39 mnemonic seed phrases using PBKDF2-HMAC-SHA512 key stretching. The resulting seed generates an Ed25519 keypair for transaction signing.

7.2 Encrypted Storage at Rest

The miner wallet file (.miner_wallet) stores keypairs encrypted with AES-256-GCM. The encryption key is derived from a user-supplied passphrase via PBKDF2-HMAC-SHA256 (480,000 iterations). Passphrases enforce strict strength validation (minimum 8 characters, uppercase, lowercase, and numeric digits). Plaintext private keys and mnemonic words are never persisted to disk.

7.3 Masked Terminal Input

Seed phrase imports and passphrase inputs utilize masked terminal handlers (getpass) to prevent shoulder-surfing and accidental terminal history exposure.

8. Cryptographic Genesis and Authorship Proof

The integrity of the protocol origin is established via a one-way preimage hash commitment. The genesis hash corresponds to H(PrivateSeed), where H denotes SHA-256. The preimage is retained exclusively by the author in cold storage. Revealing the preimage provides non-repudiable mathematical proof of initial authorship without disclosing personal legal identity.

```
Protocol:    LangIndex Protocol - Developer Adoption Index
Author:      Nael Vask
Genesis Day: 2026-11-03
State Hash:  87be4735c1f0c9f85c5e2822b893c097dfa9690ccd4bfa95374036b537090def
Signature:   3045022100e4b85c2980b1df308a287233f2b4c19559815049a4f494950b73c2ad
             2c355ad4022026dc24584cbb9d0689b9176f18370125...
```

9. Conclusion

LangIndex decouples speculative capital from price discovery by deriving market values directly from verifiable code production metrics. Through multi-source ingestion, median-based filtering, and a non-extractive token model, the protocol provides developers, enterprises, and capital allocators with an objective financial barometer of software adoption.

The protocol is currently implemented as a Phase 1 reference prototype with a local ledger, allowing the developer community to audit telemetry ingestion, PoTOP proof generation, and economic parameters before Phase 2 on-chain settlement deployment. DEK tokens in Phase 1 represent paper-trading sandbox units and do not carry real-world monetary value.

Source: https://github.com/NaelVask/langidex | Onion:
vamztshzx4gia3bzchhz5u3ozpadiwlq4fykhykmgqhxg25g2jbtntad.onion